Material das dinâmicas

Oficina **Nos trilhos do GitFlow** — branches, merges e o Modelo VIC. Tudo o que o facilitador precisa para montar o quadro colaborativo e conduzir as três dinâmicas.

O que tem aqui

1. **Como montar o quadro** — os frames de cada grupo.
2. **Dinâmica 1 — Qual trilho?**: as 12 cartas, as etiquetas e o gabarito.
3. **Dinâmica 2 — Qual foi o merge?**: as oito rodadas e o gabarito.
4. **Dinâmica 3 — A vida de uma versão**: os eventos, as peças e o gabarito.

Como montar o quadro

Use o Whiteboard do Teams, o Miro ou o Mural. Crie **um frame por grupo** para a Dinâmica 1 e outro para a Dinâmica 3, com o mesmo nome das salas simultâneas (Grupo 1, Grupo 2…).

Frame da Dinâmica 1

`main`	`develop`	`feature/*`	`release/*`	`hotfix/*`	`Não pode!`
a linha comercial	o pátio de montagem	a oficina: um vagão	a vistoria: a composição inteira	a emergência: um vagão	o que o Modelo VIC não permite

Embaixo das raias, espalhe as 12 cartas misturadas, na ordem da página 2, e uma pilha de etiquetas "sai de / vai para" em branco.

> **Importante:** faça todas as cartas na **mesma cor**, branca. Se a carta tiver a cor do trilho, o quadro entrega a resposta.

Frame da Dinâmica 3

À esquerda, os oito eventos da página 6. À direita, uma área livre com duas linhas já desenhadas, a `main` em cima e a `develop` embaixo, e um estoque de peças para copiar: linhas das cinco cores, bolinhas para as versões e etiquetas para os números.

Dinâmica 2

Não precisa de frame: as rodadas estão nos slides 29 e 30. Uma enquete com as quatro opções (Fast-forward, Merge commit, Squash e Rebase) serve para as oito rodadas.

DINÂMICA 1 · 20 MIN

Qual trilho?

Arraste cada carta para o trilho onde a situação acontece, ou para o canto "Não pode!". Depois, cole em cada carta uma etiqueta: de onde o trilho sai e para onde vai.

12 minutos nas salas · 8 minutos de conversa na volta

Os clientes vão poder simular um empréstimo no app
01

Desde ontem, o app em produção cobra a tarifa em dobro
02

A versão 2.4 vai para a homologação; daqui em diante, só correção
03

A versão que os clientes usam hoje
04

Onde se juntam as novidades prontas para a próxima versão
05

Na homologação da 2.4, um texto da tela está errado
06

Relatório novo que a diretoria pediu para o próximo trimestre
07

Erro no cálculo de juros em produção: precisa sair hoje
08

A 2.4 foi aprovada na homologação
09

Duas equipes, duas novidades diferentes, ao mesmo tempo
10

A correção urgente de ontem também precisa estar na próxima versão
11

Alterar direto a versão em produção, sem revisão, porque é rapidinho
12

Etiquetas para copiar

sai de: _____
vai para: _____

sai de: _____
vai para: _____

sai de: _____
vai para: _____

sai de: _____
vai para: _____

sai de: _____
vai para: _____

sai de: _____
vai para: _____

sai de: _____
vai para: _____

sai de: _____
vai para: _____

DINÂMICA 1 · GABARITO

As respostas

#	Carta	Trilho	Sai de → vai para	Por quê
01	Os clientes vão poder simular um empréstimo no app	`feature/*`	develop → develop	vagão novo: vai para a oficina
02	Desde ontem, o app em produção cobra a tarifa em dobro	`hotfix/*`	main → main e develop	vagão com defeito no trem comercial
03	A versão 2.4 vai para a homologação; daqui em diante, só correção	`release/*`	develop → main e develop	a composição entrou na vistoria
04	A versão que os clientes usam hoje	`main`	permanente	o trem comercial, com passageiros
05	Onde se juntam as novidades prontas para a próxima versão	`develop`	permanente	o pátio: a próxima composição
06	Na homologação da 2.4, um texto da tela está errado	`release/*`	corrige na própria release	na vistoria, só se conserta
07	Relatório novo que a diretoria pediu para o próximo trimestre	`feature/*`	develop → develop	vagão novo, sem pressa de emergência
08	Erro no cálculo de juros em produção: precisa sair hoje	`hotfix/*`	main → main e develop	não dá para esperar a próxima composição
09	A 2.4 foi aprovada na homologação	`release/*`	vai para a main e volta para a develop	fim da vistoria: linha comercial e pátio
10	Duas equipes, duas novidades diferentes, ao mesmo tempo	`feature/*`	uma para cada equipe	cada vagão na sua oficina
11	A correção urgente de ontem também precisa estar na próxima versão	`develop`	a volta do hotfix	não é trilho novo: o conserto volta para o pátio
12	Alterar direto a versão em produção, sem revisão, porque é rapidinho	`Não pode!`	—	ninguém mexe no trem comercial sem autorização

Para a conversa na volta

- **Carta 06:** por que não abre uma feature? A vistoria já começou: o conserto é feito na própria release.
- **Carta 11:** que trilho é esse? Nenhum novo: é a volta do hotfix para o pátio, a develop. Na parte 2, ela ganha o nome de merge back (Diretriz 5).
- **Carta 12:** por que não pode? A main é protegida: ninguém mexe no trem comercial sem autorização (Diretriz 7).

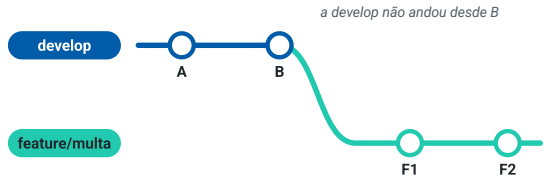
DINÂMICA 2 · 15 MIN

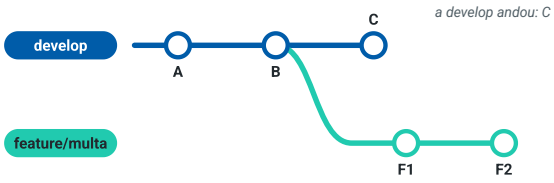
Qual foi o merge?

Uma rodada por vez. A turma vota na enquete; depois, 30 segundos de conversa.

Opções da enquete: Fast-forward · Merge commit · Squash · Rebase

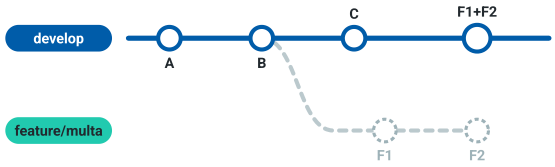
Os dois "antes"



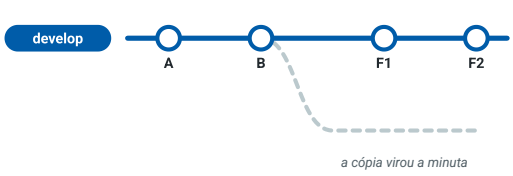


Rodadas 1 a 4: qual merge produziu cada "depois"?

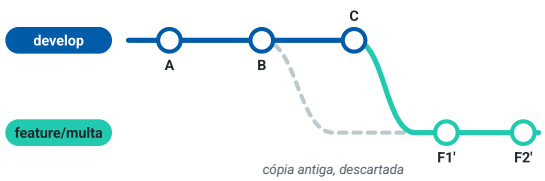
1



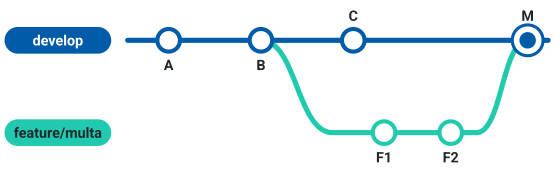
2



3



4



Rodadas 5 a 8: qual você escolheria?

5 Quero que o histórico mostre quando a cláusula de multa entrou e todas as versões dela até ficar pronta.

6 A minha cópia tem 14 versões chamadas "ajuste", "ajuste 2" e "agora vai". Quero que entre só o resultado.

7 A develop andou muito desde que comecei. Antes de pedir a aprovação, quero refazer o meu trabalho em cima da versão mais recente.

8 Ninguém mexeu na develop desde que comecei. Quero juntar sem criar versão nova.

DINÂMICA 2 · GABARITO

As respostas

#	Rodada	Resposta	Por quê
1	desenho 1	**Squash**	F1 e F2 viraram uma versão só
2	desenho 2	**Fast-forward**	linha reta e sem C: a minuta não tinha andado
3	desenho 3	**Rebase**	F1' e F2', versões novas saindo de C
4	desenho 4	**Merge commit**	a versão de junção M
5	Quero que o histórico mostre quando a cláusula de multa entrou e todas as versões dela até ficar pronta.	**Merge commit**	guarda a cópia e a junção
6	A minha cópia tem 14 versões chamadas "ajuste", "ajuste 2" e "agora vai". Quero que entre só o resultado.	**Squash**	passa a limpo
7	A develop andou muito desde que comecei. Antes de pedir a aprovação, quero refazer o meu trabalho em cima da versão mais recente.	**Rebase**	refaz a própria cópia; merge commit da develop para a cópia também resolve, e é o certo se a cópia é compartilhada
8	Ninguém mexeu na develop desde que comecei. Quero juntar sem criar versão nova.	**Fast-forward**	a minuta não andou

O que observar: a rodada 2 é a pegadinha. O desenho não tem C porque partiu do "antes" em que a minuta não andou; se alguém notar, elogie. Na rodada 7, reforce: rebase só na própria cópia, nunca numa linha que outras pessoas usam.

Os quatro merges, lado a lado

Tipo	O histórico mostra	No contrato	Cuidado
Fast-forward	linha reta, sem a cópia	a cópia vira a minuta	só quando a minuta não andou
Merge commit	tudo, com a versão de junção	a cláusula com todas as versões	histórico trançado
Squash	uma versão por entrega	passar a limpo	perde o passo a passo
Rebase	linha reta, com versões novas	refazer sobre a minuta atual	reescreve a história

DINÂMICA 3 · 25 MIN

A vida de uma versão

O **Contratos Online** é um sistema fictício. Em produção está a versão `v2.3.0.97`. Desenhem as linhas do tempo no quadro, na ordem dos eventos, e numerem cada versão com o número do evento.

15 minutos nas salas · 10 minutos de conversa na volta

Os eventos

1. A equipe A começa a assinatura digital.
2. A equipe B começa o alerta de vencimento.
3. A assinatura digital fica pronta; o PR é aprovado e o Integrador conclui o merge.
4. A versão 2.4.0 vai para homologação, com a assinatura digital. O alerta ainda não está pronto.
5. Na homologação, aparece um erro de texto na tela de assinatura; é corrigido.
6. A homologação aprova: a versão entra em produção (build 118).
7. No dia seguinte, a multa por atraso sai calculada errada em produção. A correção sai no mesmo dia (build 121).
8. O alerta de vencimento fica pronto e entra para a próxima versão.

Respondam também

1. Que branches temporárias existiram, e com que nome?
2. Que números de versão a main recebeu?
3. Em que versão o alerta de vencimento chega aos clientes?
4. O que aconteceria sem o merge back do evento 7?
5. Que tipo de merge você usaria no evento 3?

As peças



linhas: main · develop · feature · release · hotfix — versão — número da versão (só na main)

Antes de apresentar, confiram: cada branch temporária saiu do lugar certo? A release e o hotfix voltaram para os dois destinos? Os números de versão estão só na main?

DINÂMICA 3 · GABARITO

A vida de uma versão



Pergunta	Resposta
Que branches temporárias existiram, e com que nome?	`feature/assinatura` `feature/alerta` `release/2.4.0` `hotfix/2.4.1`
Que números de versão a main recebeu?	`v2.4.0.118` (evento 6) e `v2.4.1.121` (evento 7)
Em que versão o alerta de vencimento chega aos clientes?	Na 2.5.0, a próxima release: MINOR, porque é novidade que não muda o que já existia
O que aconteceria sem o merge back do evento 7?	A 2.5.0 sairia da develop sem a correção e voltaria a calcular a multa errada
Que tipo de merge você usaria no evento 3?	Aberta. No GitFlow original, merge commit; squash, se a cópia tiver muitas versões pequenas. A política do repositório decide

Erros mais comuns

- **O hotfix saindo da develop:** emergência sai da main, que é onde está o que o cliente usa.
- **O alerta entrando na release/2.4.0:** a versão já estava em homologação; o alerta espera a próxima.
- **Um destino esquecido:** a release e o hotfix voltam para a main e para a develop.
- **Número de versão na develop:** o número só vai na main.